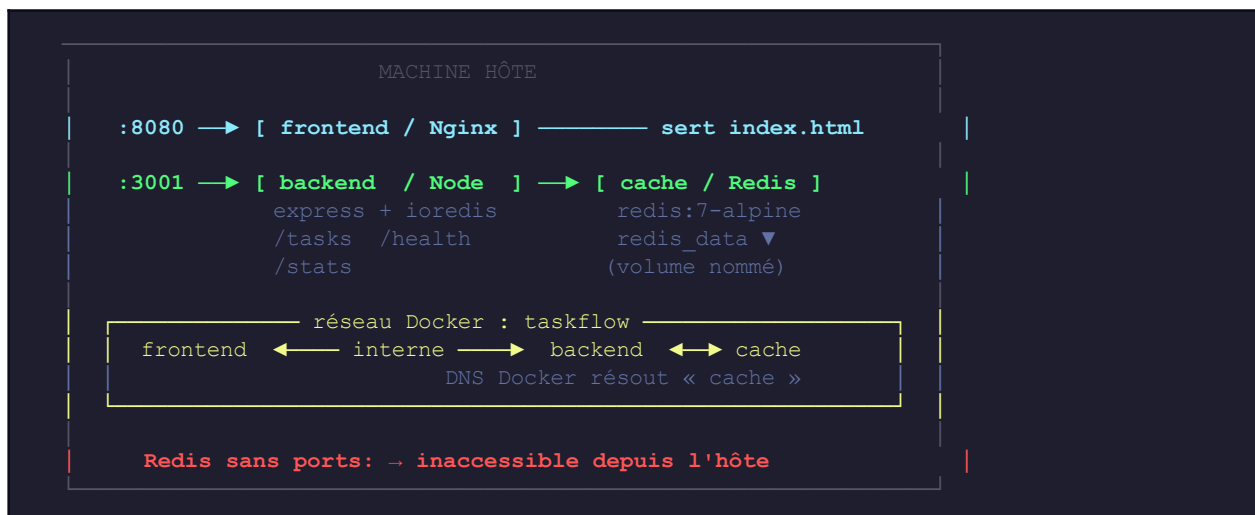


TP3 — TaskFlow

Dockerfiles · Docker Compose · Réseaux · Volumes · .env

Ducarois Mathis

Architecture de la stack TaskFlow



Service	Image	Port hôte	Port interne	Rôle
frontend	nginx:alpine	:8080	:80	Sert index.html via Nginx
backend	node:20-alpine	:3001	:3001	API REST (Express + ioredis)
cache	redis:7-alpine	—	:6379	Stockage des tâches. Non exposé !

Étape 1

Dockerfile backend — API Node.js

Fichier à créer : `backend/Dockerfile`. L'image doit permettre de lancer l'API Node.js (Express + ioredis).

```
FROM node:20-alpine

WORKDIR /app

# Optimisation du cache : dépendances en premier
COPY package*.json ./
RUN npm install --omit=dev

# Code source copié APRÈS les dépendances
COPY server.js ./

# Utilisateur non-root fourni par node:20-alpine
USER node

EXPOSE 3001
CMD ["node", "server.js"]
```

Explications — backend/Dockerfile

Instruction	Pourquoi
<code>FROM node:20-alpine</code>	node:20-alpine utilise Alpine Linux (~50 MB). L'image de base officielle node:20 (Debian) dépasse 1 Go. Alpine suffit largement pour exécuter une API Express/ioredis.
<code>COPY package*.json → npm install</code>	On copie package.json AVANT server.js. Si server.js change mais que les dépendances restent identiques, la couche npm install sera CACHED → rebuild instantané. Sans cette astuce, npm install tourne à chaque modification du code.
<code>--omit=dev</code>	Exclut les devDependencies (nodemon...) de l'image finale. Image plus légère, surface d'attaque réduite. En production, on n'a pas besoin des outils de dev.
<code>USER node</code>	node:20-alpine fournit un utilisateur node (uid=1000). Basculer sur cet utilisateur avant CMD évite que le processus tourne en root. Bonne pratique de sécurité recommandée par l'énoncé.
<code>CMD ["node", "server.js"]</code>	Forme exec (tableau) plutôt que forme shell (string). Le processus node reçoit directement les signaux SIGTERM/SIGINT → arrêt propre à docker compose down.

Étape 2

Dockerfile frontend — Nginx

Fichier à créer : `frontend/Dockerfile`. Sert l'interface web statique (index.html) via Nginx.

```
FROM nginx:alpine

# Fichiers statiques vers le dossier servi par Nginx
COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80
```

Point	Explication
<code>nginx:alpine</code>	Image officielle Nginx basée sur Alpine. ~25 MB. Sert les fichiers statiques avec de très bonnes performances. Aucune dépendance Node n'est nécessaire côté frontend.
<code>COPY index.html /usr/share/nginx/html/</code>	C'est le répertoire par défaut servi par Nginx. On pourrait aussi copier un dossier complet (CSS, JS...). Si un <code>nginx.conf</code> custom est fourni, on le copie vers <code>/etc/nginx/conf.d/default.conf</code> .
<code>EXPOSE 80</code>	Nginx écoute sur le port 80 par défaut. Dans <code>docker-compose.yml</code> , ce port est mappé sur 8080 de l'hôte (8080:80).

Étape 3

docker-compose.yml — Orchestration des 3 services

```

services:

# — 1. Cache Redis — PAS de ports: —————
cache:
  image: redis:7-alpine
  restart: unless-stopped
  # Aucune directive ports: → Redis non exposé
  volumes:
    - redis_data:/data
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 5s | timeout: 3s | retries: 5
  networks:
    - taskflow

# — 2. Backend Node.js —————
backend:
  build:
    context: ./backend
    dockerfile: Dockerfile
  environment:
    APP_ENV:      ${APP_ENV:-local}
    PORT:         ${BACKEND_PORT:-3001}
    REDIS_HOST:   cache          # DNS interne Docker
    REDIS_PORT:   ${REDIS_PORT:-6379}
  ports:
    - "${BACKEND_PORT:-3001}:3001"
  depends_on:
    cache:
      condition: service_healthy # attend que Redis réponde
  networks:
    - taskflow

# — 3. Frontend Nginx —————
frontend:
  build:
    context: ./frontend
  ports:
    - "${FRONTEND_PORT:-8080}:80"
  depends_on:
    - backend
  networks:
    - taskflow

networks:
  taskflow:          # réseau partagé des 3 services
  driver: bridge

volumes:
  redis_data:        # données Redis persistées

```

Concepts clés expliqués

Concept	Explication détaillée
<code>Pas de ports: sur cache</code>	L'énoncé l'exige explicitement : Redis ne doit PAS être accessible depuis l'hôte. En omettant la section ports: sur le service cache, seuls les autres containers du réseau taskflow peuvent l'atteindre (ici uniquement le backend). → docker ps ne montrera aucun port mappé pour cache. C'est volontaire. → nc localhost 6379 depuis la machine hôte doit échouer.
<code>REDIS_HOST: cache (DNS interne Docker)</code>	Docker Compose crée un réseau DNS interne : chaque service est joignable par son nom. Le backend se connecte à Redis via host: 'cache', pas 'localhost'. → Redis('{ host: process.env.REDIS_HOST }') résout 'cache' → IP du container Redis. Ce mécanisme fonctionne tant que les deux services sont sur le même réseau.
<code>depends_on: condition:service_healthy</code>	Le healthcheck redis-cli ping tourne toutes les 5 secondes dans le container cache. Avec condition: service_healthy, le backend ne démarre qu'une fois que Redis répond PONG. Sans ça, ioredis tenterait de se connecter trop tôt et crasherait. → Plus robuste que depends_on simple qui ne vérifie que le démarrage du container.
<code>volumes: redis_data:/data</code>	Redis stocke ses données dans /data par défaut (RDB snapshot). Sans volume : docker compose down efface tout. Avec le volume nommé redis_data, les tâches persistent entre les redémarrages. → docker compose down : containers supprimés, données Redis conservées → docker compose down -v : tout supprimé, remise à zéro complète
<code>\${BACKEND_PORT:-3001} (valeurs par défaut)</code>	La syntaxe \${VAR:-valeur} lit la variable du .env (ou de l'environnement) et utilise la valeur par défaut si elle n'est pas définie. → La stack fonctionne sans .env (valeurs par défaut appliquées). → Avec .env : les valeurs du fichier prennent le dessus.

Étape 4

.env.example et .dockerignore

Fichier .env

```
# Environnement applicatif
APP_ENV=local

# Ports exposés sur la machine hôte
FRONTEND_PORT=8080
BACKEND_PORT=3001

# Redis — connexion via nom de service Docker
REDIS_HOST=cache
REDIS_PORT=6379
```

Fichier .dockerignore

```
node_modules/
.env
.env.*
.git/
*.log
dist/
build/
Dockerfile*
docker-compose*.yaml
*.md
.vscode/ .idea/
```

Règle	Raison
node_modules/	Binaires compilés pour l'OS local (macOS/Windows) ≠ Alpine Linux. npm install dans le Dockerfile recrée les bons binaires. Copier le node_modules local casserait l'app.
.env / .env.*	Contient les mots de passe Redis. Une fois dans l'image, tout développeur avec docker history peut les lire. Ne JAMAIS inclure dans le contexte de build.
.git/	Historique complet du dépôt inutile en runtime. Peut contenir d'anciens secrets. Alourdit le contexte de build (parfois plusieurs centaines de MB).
dist/ build/	Si l'application a une étape de build, le dossier local peut être périmé ou incompatible. On préfère que le build se fasse dans le container. À retirer si votre Dockerfile en a besoin.

Commandes de vérification

```
# 1. Lancer toute la stack
docker compose up --build

# 2. Vérifier les 3 services
docker compose ps
#   NAME                STATUS      ← Redis OK
#   tp3-cache-1          healthy
#   tp3-backend-1        running    ← API OK
#   tp3-frontend-1       running    ← Nginx OK

# 3. Tester l'API
curl http://localhost:3001/health
# → { "status": "ok", "service": "taskflow-api" }

curl http://localhost:3001/tasks
# → [] (vide au départ)

curl -X POST http://localhost:3001/tasks \
  -H "Content-Type: application/json" \
  -d '{"title":"Conteneuriser TaskFlow","priority":"high"}'
# → { id, title, priority, done, createdAt }

# 4. Tester le frontend
open http://localhost:8080
# → Interface TaskFlow dans le navigateur

# 5. Vérifier que Redis N'EST PAS exposé
# docker compose ps → aucun port mappé sur cache
# docker inspect tp3-cache-1 | grep PortBindings
# → {} (vide = pas d'exposition)

# 6. Tester la persistance
# → Ajouter une tâche via l'interface
docker compose down
docker compose up
# → La tâche est toujours présente

# 7. Reset complet (efface le volume Redis)
docker compose down -v
```